

10 Proven Agents & Automations Prompts

Copy any prompt, fill the [BRACKETS], send. Mined from 500+ hours of top Claude builders.

1

Workflow Before Agent

~50% of business automations need no AI at all and most need only a deterministic workflow — agents add latency, cost, and inconsistency when unnecessary.

I want to automate this: [DESCRIBE THE TASK]. Before we build anything, walk me through this decision: do I need to be in the loop every time (a simple assistant), are the steps fully logic-based (a plain workflow with no AI), is the order of operations fixed (an AI workflow), or is it genuinely unpredictable (a true agent)? Recommend the simplest option that actually solves it.

2

Wireframe the Automation First

Jumping straight into the builder creates messy, over-complicated automations and hours of debugging; wireframing is the Lego instruction manual.

Before building, help me wireframe this automation on paper. Ask me: what triggers it, what the trigger data looks like, how data moves step to step, where I need conditional logic or an AI decision, where I need an API call or RAG, and what integrations are involved. Map the whole thing out first — more than half our time should be planning, not building.

3

Define What 'Done' Looks Like

Agents need a clear finish line or they overcomplicate, loop, and waste tokens — defining 'done' concretely keeps them on track.

Here's the task: [TASK]. Don't start until 'done' is crystal clear. The finished state is: [EXACT DELIVERABLE — e.g. 'exactly 75 rows in a spreadsheet with name, company, email, and profile link']. Once you have produced exactly that, stop. Confirm you understand the finish line before you begin.

4

The 6-Section Agent System Prompt

A complete, proven skeleton for any tools-agent — the tool section is the most important part, and examples reduce ambiguity.

Help me write a system prompt for an agent using this 6-section structure: (1) Role — 'You are a [X] agent designed to [Y]; your goal is [Z]'; (2) Context — the input it receives every run; (3) Tools — a numbered list, each with 'use this tool when...' plus any ordering rules; (4) Rules — conditional 'if X then Y' instructions; (5) Examples — input/actions/output, only for mistakes it repeats; (6) Final notes — current date/time, format rules. Keep it lean, no filler.

5

Reactive Prompting

Proactive mega-prompts make debugging impossible; reactive prompting isolates every cause and prevents bloated, confused prompts.

Let's build this agent reactively, not all at once. Start it with NO system prompt and just one tool. We'll test it, watch what it does, then add exactly one sentence to fix what we saw – and only ever change one thing per cycle so I know exactly what fixed or broke it. Don't paste a giant pre-written prompt.

6

Orchestrator + Sub-Agent Architecture

A delegation-only orchestrator with modular sub-agents stays token-efficient, easy to debug, and reusable — coordination breaks down past ~5 sub-agents.

Design this as an orchestrator plus specialized sub-agents. The orchestrator gets a short 3-sentence system prompt – 'You are a delegator; you do not do the work yourself, you route each request to the correct sub-agent.' Build each sub-agent (e.g. [research], [email], [content]) as its own modular, reusable unit with its own focused prompt and tools. Never give one agent more than ~10 tools; cap sub-agents at ~5.

7

Always Inject the Current Date

The single most common silent agent bug — agents need today's date to resolve relative dates correctly.

This agent handles dates and scheduling. Add this to its system prompt: 'Today's date and time is [CURRENT DATE], in [TIME ZONE].' Without it, agents resolve 'tomorrow' or 'next week' to the wrong year and create events in the past. Confirm the date is wired in dynamically, not hard-coded.

8

Self-Healing Loop

Agentic automations should be self-correcting — read the error, fix the tool, update the workflow so the failure never happens again.

Give this agent a self-healing loop with three layers: (1) fix the script — if it errors, read the error, edit its own code, and retry; (2) fix the prompt — if it spots a way to improve its own instructions, rewrite them; (3) memory log — after each run, overwrite a single status file with what happened so the next run knows the state. When it hits an error it shouldn't quit — it should try other approaches and update itself so the failure never recurs.

9

Human-in-the-Loop Before Publishing

Approval gates make agents reliable before they touch the public; the latest-version tracking prevents revision loops from losing the current draft.

For anything this agent posts publicly or sends externally, add a human approval gate. Route the draft to me for review, let me reply with free-text feedback, classify my reply as approval or revision, and if it's a revision send it to a reviser agent and loop back to me. Always keep the latest version tracked so revisions stack correctly. Never auto-publish.

10

Schedule It as a 24/7 Employee

Scheduled tasks run Claude agentially on a schedule; the one-file-per-task log gives stateless runs memory without append-log bloat.

Turn this working automation into a scheduled task. Set it to run on [CRON SCHEDULE]. Structure the prompt as: 'Before doing your job, read last-run.md. Do your main task. Then overwrite last-run.md with current status, what you did, and any known issues for the next run.' Test it with 'run now' a few times before going live, and have it message me if it fails.